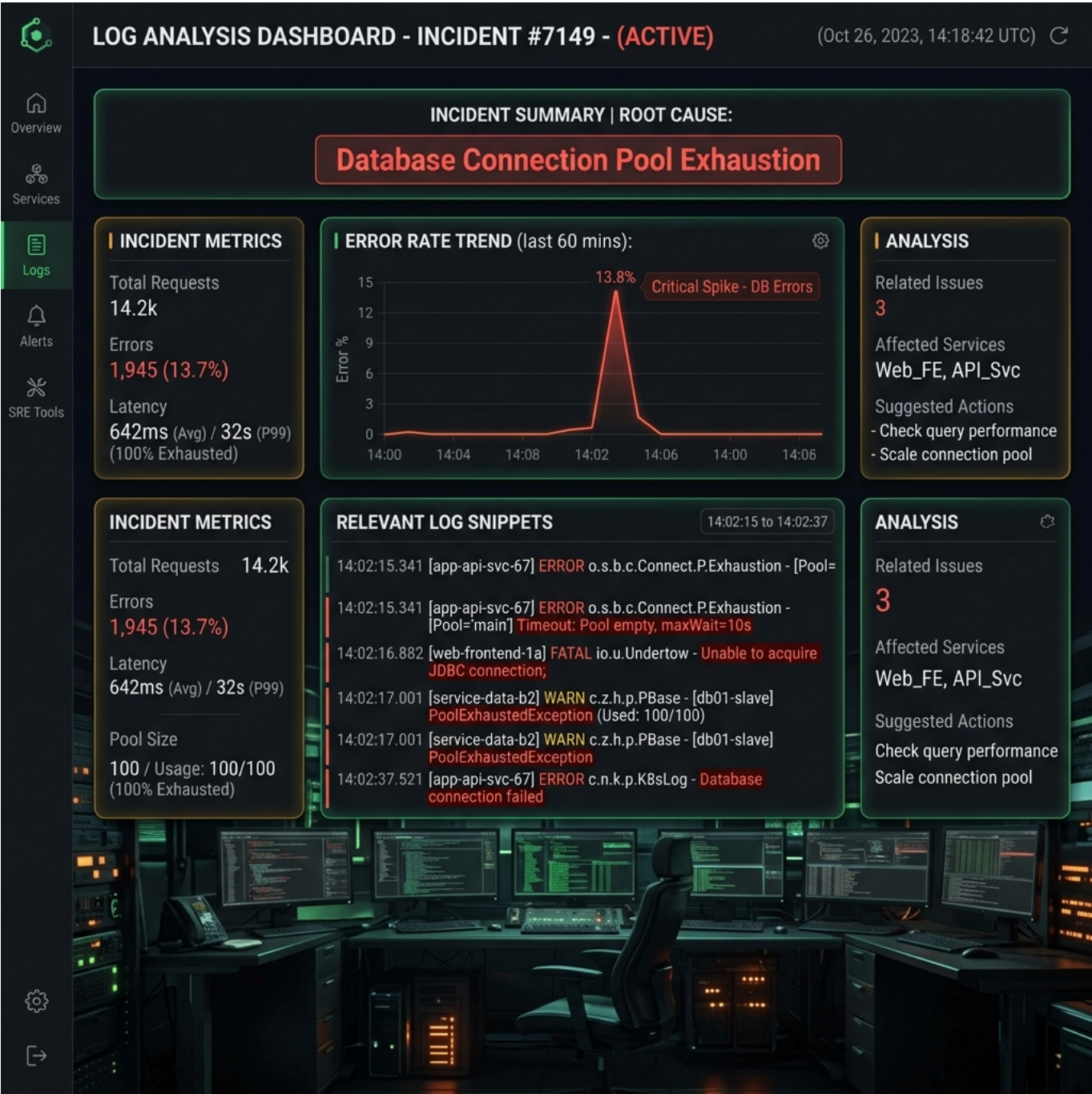


Incident Response Report

1. Log Analysis & Diagnosis

Root Cause: Database connection pool exhaustion
Confidence: 90%

Details:
The logs show a pattern of increasing connection pool usage, eventually reaching the limit, which leads to timeouts and errors in various services, including rebalance_service. The session leaks and slow queries also exacerbate the issue.



Incident Response Report

2. Technical Solution Research

Solutions Evaluated:

- Identifying and Fixing SQLAlchemy Session Leaks: Ensure every session is closed using context managers or try/finally blocks. Common in long-running background tasks or rebalance workflows.
- Python Context Manager Patterns for DB Sessions: Use `@contextmanager` to wrap DB sessions. Ensure `session.close()` is always called.

3. Final Remediation Plan

Selected Solution: Identifying and Fixing SQLAlchemy Session Leaks

Steps:

1. Review the codebase for areas where database sessions are not properly closed, focusing on long-running background tasks and rebalance workflows.
2. Implement context managers or try/finally blocks to ensure sessions are closed after use.
3. Perform thorough testing of the changes, including load testing to simulate the conditions leading to connection pool exhaustion.
4. Deploy the updated code to the production environment, ensuring that the deployment process includes rollback capabilities.

Incident Response Report

OpsWorks
Remediation

Dashboard

Incident Management

Change Requests

Reports

Settings

Remediation Plan: High Latency Issue - Production DB

Incident ID: INC-2023-10-14-A

Title: Remediate High API Latency & Database Connection Errors

Summary

Status: Approved

Assigned To: Engineering Team

Created By: Jane Doe

Objectives

Resolve the 503 errors and high latency in the API by optimizing database connection management.

Remediation Steps

1. ☒ Analyze Metrics: Review current database and application metrics to confirm latency source.

2. ☒ Prepare Configuration: Create new PgBouncer configuration and review update commands.

3. ☐ Update DB_POOL_MAX:

Modify the DB_POOL_MAX setting from 20 to 50 on the central application configuration.

Requires dynamic reload of configuration.

Assignee: Mark J.

Status: Pending

4. ☐ Deploy PgBouncer:

Deploy PgBouncer as a connection pooler layer between the application servers and the Postgres database.

Verify health checks for the PgBouncer pods.

Assignee: Sarah K.

5. ☐ Traffic Cutover: Reroute application database connections to the new PgBouncer endpoint.

6. ☐ Validate API Health:

Monitor error rates, response times, and connection count using Prometheus and Grafana.

Perform end-to-end API tests for critical endpoints.

7. ☐ Monitor & Review: Continue monitoring for 24 hours. Hold a post-incident review meeting.

Rollback Plan

If latency worsens, revert the DB_POOL_MAX to 20.

If PgBouncer fails, cut back database traffic directly to the Postgres server.

Edit Plan

Start Execution

Approve